Security Review

Cantina Managed

SKY: SPARK BOOSTED VAULT

Cantina Managed review by:
Christoph Michel, Lead Security Researcher
M4rio.eth, Lead Security Researcher

CANTINA

August 14, 2026

Contents

1 Introduction **2**
- 1.1 About Cantina . . . 2
- 1.2 Disclaimer . . . 2
- 1.3 Risk assessment . . . 2
  - 1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**
- 2.1 Scope . . . 3

3 Findings **4**
- 3.1 Low Risk . . . 4
  - 3.1.1 Rounded partial withdrawals can delete unpaid principal . . . 4
- 3.2 Informational . . . 4
  - 3.2.1 `maxDeposit()` can quote non-zero deposits that always revert . . . 4
  - 3.2.2 Deployment silently truncates `term` and `cliff` . . . 5
  - 3.2.3 `maxLiability()` can be lower than `totalPrincipal` . . . 6
  - 3.2.4 Minor issues, Typos & Documentation . . . 6

1 Introduction

1.1 About Cantina

Cantina is a security platform that pairs a world-class network of security researchers with agentic AI to help teams building onchain find and fix real risk. Through managed reviews, competitions, and bounties, Cantina secures protocols before vulnerabilities can be exploited.
Learn more at cantina.security

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

From Aug 11th to Aug 13th the Cantina team conducted a review of spark-boosted-vault on commit hash f2963516. The team identified a total of **5** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	1	0	1
Gas Optimizations	0	0	0
Informational	4	4	0
Total	**5**	**4**	**1**

The Cantina Managed team reviewed Sky's spark-boosted-vault holistically on commit hash 26dcfaf9 (tag v1.0.0) and concluded that all findings were addressed and no new vulnerabilities were identified.

2.1 Scope

The security review had the following components in scope for spark-boosted-vault on commit hash f2963516:

```
src/SparkBoostedVault.sol
```

3 Findings

3.1 Low Risk

3.1.1 Rounded partial withdrawals can delete unpaid principal

Severity: Low Risk.

Context: SparkBoostedVault.sol#L503-L520.

Description: `SparkBoostedVault._withdraw(...)` correctly closes a position when a ceiling-rounded withdrawal exhausts either its shares or principal, because a live position requires both components to remain nonzero. However, the branch promotes only `sharePortion_` and `principalPortion_` to their full position balances while leaving `assets_` (the requested `assetsOut`) unchanged.

Consequently, the complete position and its aggregate accounting are deleted, but the user receives only the originally requested partial output rather than the full `withdrawable_` claim:

```
SparkBoostedVault.withdraw(positionId_, assetsOut, recipient_)
└─ SparkBoostedVault._withdraw(...)
   ├─ sharePortion_ >= shares_ || principalPortion_ >= principal_
   ├─ delete complete position
   ├─ sharePortion_ = shares_
   ├─ principalPortion_ = principal_
   └─ SparkBoostedVault._pushAsset(recipient_, assetsOut)
      // assetsOut was not promoted to withdrawableAssets
```

The unpaid residual loses all associated accounting and becomes vault surplus, which contradicts the documented guarantee that deposited principal is always withdrawable.

Example: Assume token base units, `chi = 1.5 RAY`, no subsequent `chi` change, and a position that remains before its cliff:

1. Depositing 100 assets creates `floor(100 RAY / 1.5 RAY) = 66` shares and records 100 principal.
2. Since no yield is vested, `withdrawable_ = principal_`.
3. After 65 withdrawals of 1 asset, ceiling rounding burns one share and one principal unit per call, leaving 1 share and 35 principal.
4. On the 66th withdrawal, `sharePortion_ = ceil(1 * 1 / 35) = 1`, which exhausts the final share, while `principalPortion_ = ceil(35 * 1 / 35) = 1`.
5. The full-exit branch deletes the position and overwrites `principalPortion_` with 35, but `_pushAsset(...)` transfers only the requested 1 asset.
6. The user receives only 66 of their original 100 principal assets, while the remaining 34 assets become unaccounted vault surplus.

Recommendation:

When either rounded component exhausts the position, promote `assets_` to the full current claim alongside the existing full-burn assignments:

```
  if (sharePortion_ >= shares_ || principalPortion_ >= principal_) {
      positionIdSet_.remove(positionId_);

      delete $.positions[positionId_];

      sharePortion_     = shares_;
      principalPortion_ = principal_;
+     assets_           = withdrawable_;
  }
```

Note that this would change an implicit invariant that `withdraw(assets)` calls always transfer *exactly* `assets` to `withdraw(assets)` calls always transferring *at least* `assets`.

Sky: Acknowledged. We prefer the invariant that `withdraw(assets)` calls always transfer exactly `assets`, and any rounding dust is lost. For almost every withdrawal other than "instant" withdrawals, the loss will be of interest.

Cantina Managed: Acknowledged.

3.2 Informational

3.2.1 `maxDeposit()` can quote non-zero deposits that always revert

Severity: Informational.

Context: SparkBoostedVault.sol#L273-L283.

Description: `SparkBoostedVault.maxDeposit()` returns the positive difference between `maxLiabilityCap` and the reported current liability, but it does not verify that this remaining asset capacity is sufficient to mint at least one position share.

`SparkBoostedVault._deposit(...)` subsequently calculates `shares_ = assets_ * RAY / chi` with downward rounding and reverts with `ZeroDeposit()` when the result is zero. Consequently, `maxDeposit()` can advertise a positive deposit that is guaranteed to revert even when the caller has sufficient balance and allowance and the vault state remains unchanged.

This violates a natural expected invariant:

```
maxDeposit() > 0 => deposit(maxDeposit()) does not revert
```

Example:

Assume token base units, an empty vault, `chi = 1.5 RAY`, and `maxLiabilityCap = 1`:

1. `maxLiability()` returns zero because `totalShares = 0`.
2. `maxDeposit()` returns `1 - 0 = 1`.
3. A funded and approved user calls `deposit(1)`.
4. The cap check passes because `0 + 1 <= 1`.
5. The deposit calculates `shares_ = floor(1 * RAY / 1.5 RAY) = 0`.
6. The transaction reverts with `ZeroDeposit()`.

Recommendation: Consider changing `maxDeposit()` to be conservative and take the zero-shares revert rounding in `deposit` into account, i.e., return zero whenever the remaining capacity cannot mint at least one share:

```
function maxDeposit() external view override returns (uint256) {
    uint256 maxLiability_    = maxLiability();
    uint256 maxLiabilityCap_ = _getVaultStorage().maxLiabilityCap;

    if (maxLiability_ >= maxLiabilityCap_) return 0;

    uint256 remainingCapacity_ = maxLiabilityCap_ - maxLiability_;
    uint256 minDeposit_        = Math.ceilDiv(nowChi(), RAY);

    // must be at least minDeposit to mint at least 1 share and avoid revert in
    ↪  deposit
    // otherwise return 0, indicating vault is at capacity
    return remainingCapacity_ >= minDeposit_ ? remainingCapacity_ : 0;
}
```

Note: The mentioned invariant might still not hold when `maxDeposit()` returns very large `asset` values that would overflow the math in `deposit` but these asset values are never reached in practice for the expected tokens.

Sky: Fixed in PR 11.

Cantina Managed: Fix verified.

3.2.2 Deployment silently truncates `term` and `cliff`

Severity: Informational.

Context: Deploy.s.sol#L54-L73.

Description: The deployment script reads `term` and `cliff` as arbitrary-width integers, then explicitly casts them to `uint64`:

```
uint64 term  = uint64(inputConfig.readUint(".term"));
uint64 cliff = uint64(inputConfig.readUint(".cliff"));
```

Values larger than `type(uint64).max` are truncated instead of reverting. The post-deployment checks do not detect this because they compare the proxy values against the already-truncated locals:

```
require(proxy.term() == term);
require(proxy.cliff() == cliff);
```

For example, a configured term of `2^64 + 30 days` is deployed as `30 days`, and the check still passes.

Recommendation: Validate the original values before casting:

```
uint256 termInput  = inputConfig.readUint(".term");
uint256 cliffInput = inputConfig.readUint(".cliff");

require(termInput <= type(uint64).max, "term exceeds uint64");
require(cliffInput <= type(uint64).max, "cliff exceeds uint64");

uint64 term  = uint64(termInput);
uint64 cliff = uint64(cliffInput);
```

Sky: Fixed in PR 8.

Cantina Managed: Fix verified.

3.2.3 `maxLiability()` can be lower than `totalPrincipal`

Severity: Informational.

Context: SparkBoostedVault.sol#L273-L283, SparkBoostedVault.sol#L451-L464.

Description: The vault stores the full deposited amount as `principal`, while `maxLiability()` values only the rounded-down shares:

```
shares = assets * RAY / chi; // rounds down
principal = assets;
maxLiability = totalShares * nowChi() / RAY;
```

For example, at `chi = 1.5 RAY`, depositing `100` records `principal = 100` but mints `66` shares. The share-valued liability is therefore:

```
floor(66 * 1.5) = 99
```

while `100` remains withdrawable as principal.

The rounding direction is fine. The documentation and accounting semantics are ambiguous because `maxLiability()` can be lower than the full claimable principal.

Recommendation: Document that `maxLiability()` is a share-value metric and to be mindful of rounding.

Sky: Fixed in PR 13.

Cantina Managed: Fix verified.

3.2.4 Minor issues, Typos & Documentation

Severity: Informational.

Context: *(See each case below)*.

Description:

1. SparkBoostedVault.sol#L518 - the `Withdraw` event should contain recipient.
2. SparkBoostedVault.sol#L433 - `@notice Returns the vesting multiplier [ray] for the user's current position.` ⇒ `@notice Returns the vesting multiplier [ray] for the user's _positionId`.
3. SparkBoostedVault.sol#L54-L64 Pack `maxVsr` and `minVsr` as `uint96`.
4. SparkBoostedVault.sol#L444-L447: Could directly use the `maxDeposit()` helper for the check `assets_ <= maxDeposit`. The event would then be adjusted to report `MaxLiabilityCapExceeded(assets, maxDeposit)`.

Recommendation: Fix the above issues.

Sky: Fixed in commit 199c8612.

Cantina Managed: Fix verified for points 1, 2 and 4. 3 has been acknowledged.